Assignment 1: Reinforcement Learning

Jianong Xu | October 30, 2026

GitHub: **EvelynXuNU** · Repository: **iems_HW1** · https://github.com/EvelynXuNU/iems_HW1 (**public**).

This report answers all three required parts. Shuttle values are discounted **costs** (smaller is better); game plots and summaries use undiscounted scores. Policy-gradient evaluation always uses **500 frozen, stochastic-policy episodes**. The required Pong runs did not learn winning policies; that limitation is reported alongside mathematical and behavioral validation. Full traces, raw data, source, checkpoints and expanded technical notes are available in the repository.

1. Value and policy iteration: model and solution

Use the assigned $K = 15, c_f = 100, c_h = 2, \gamma = 0.95$, iid arrivals $A_t \sim \text{Unif}\{1, 2, 3, 4, 5\}$, and states $s \in \{0, \dots, 200\}$. The state is the queue **before** dispatch. Action $a = 0$ waits; $a = 1$ dispatches one shuttle, including a partially filled shuttle. Service occurs before holding cost and new arrivals:

$$q(s, a) = \max(s - 15a, 0), \qquad c(s, a) = 100a + 2q(s, a), \qquad s' = \min(200, q(s, a) + A_t).$$

Explicit assumptions. Excess arrivals are rejected without extra penalty or an outside backlog. One shuttle is available each epoch; travel time imposes no additional state. Arrivals are independent of actions. For $T = 500$, decisions occur at $t = 0, \dots, 499$, with $J_{500} = 0$: no forced terminal clearing. Exact action ties choose wait. Under a reward convention, $r = -c, V = -J$.

(a) Enumeration/backward induction. Evaluate both actions and all five arrival outcomes at every state and time:

$$J_t(s) = \min_{a \in \{0,1\}} \left[100a + 2q(s, a) + \frac{0.95}{5} \sum_{k=1}^{5} J_{t+1}(\min(200, q(s, a) + k)) \right].$$

This enumerates adaptive action/outcome choices while reusing repeated state-time subproblems: $500(201)(2)(5) = 1{,}005{,}000$ outcome evaluations. Enumerating predetermined action sequences would instead solve an open-loop problem. A separate literal decision-tree oracle checks the recursion on a small MDP.

(b) Infinite-horizon value iteration. Use the same Bellman operator $\mathcal{T}$, initialize $J^{(0)} = 0$, and update synchronously until the returned vector has residual $\|\mathcal{T}J - J\|_\infty \le 5 \times 10^{-10}$. Contraction gives

$$\|J - J^*\|_\infty \le \frac{\|\mathcal{T}J - J\|_\infty}{1 - \gamma}.$$

After **487 updates**, residual 4.8203×10^{-10} implies a value-error bound 9.6406×10^{-9}, apart from numerical rounding.

(c) Policy iteration. Starting from all-wait, solve $(I - 0.95P_\pi)J_\pi = c_\pi$, then greedily improve all 201 actions. Six evaluations change 196, 28, 22, 7, 1 and 0 actions. No threshold is imposed:

$$\boxed{\pi^*(s) = 0 \text{ for } s \le 12, \qquad \pi^*(s) = 1 \text{ for } s \ge 13.}$$

1.1 Required plots and numerical results

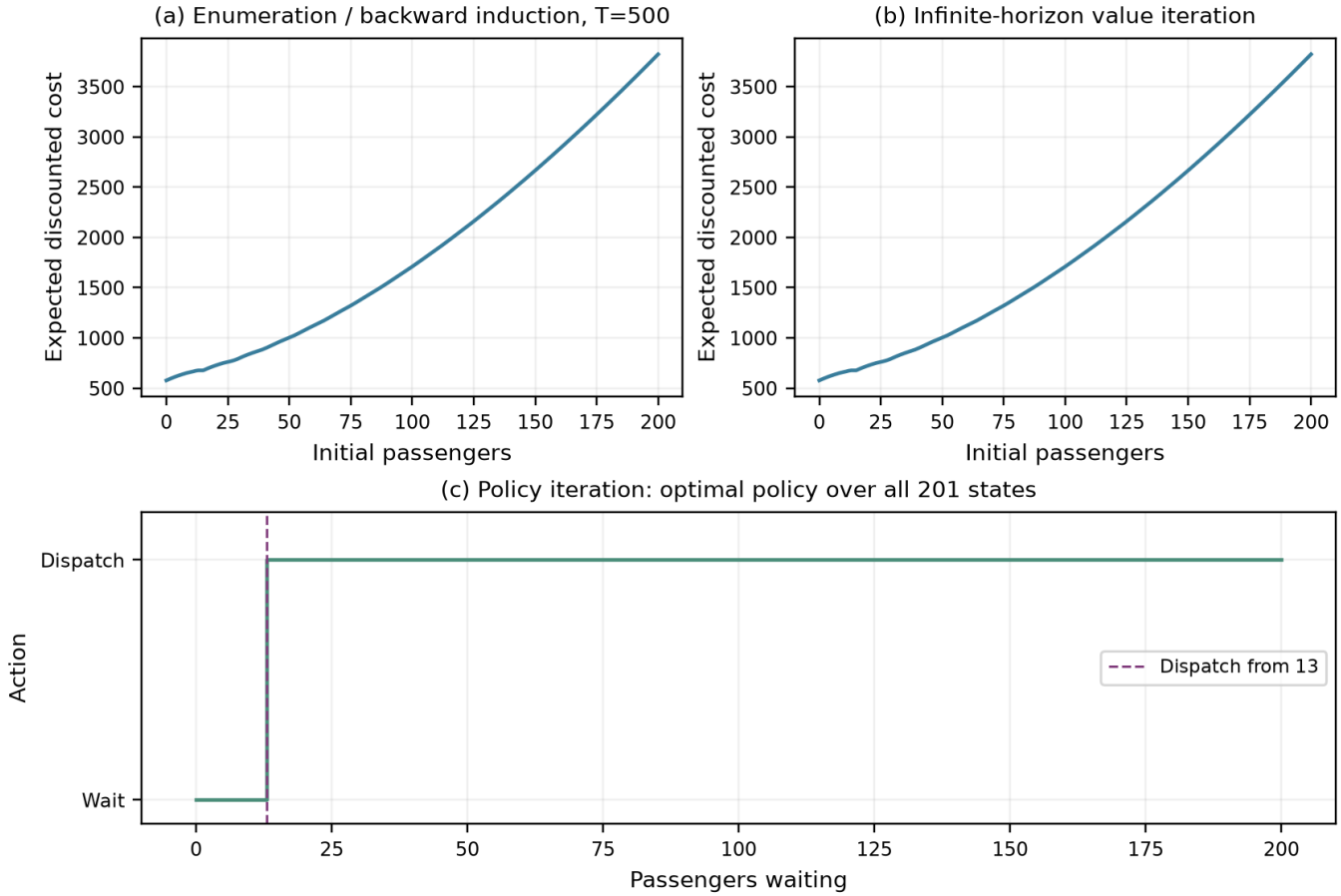


Figure 1. (a) Optimal time-zero cost with 500 decisions; (b) infinite-horizon value-iteration cost; (c) policy-iteration action over all queue states. The finite/infinite curves differ by at most 4.94×10^{-9}, so they look identical. All axes retain the full required state range.

Initial queue	Finite cost at time 0	Infinite optimal cost	Action
0	576.670168	576.670168	Wait
12	672.892163	672.892163	Wait
13	676.670168	676.670168	Dispatch
15	676.670168	676.670168	Dispatch
50	1001.527732	1001.527732	Dispatch
200	3821.586477	3821.586477	Dispatch

Dispatching before a full shuttle balances fixed service cost with future waiting cost. At 12, waiting is cheaper by 3.778004; at 13, dispatching is cheaper by 4.099663. The minimum action gap is far larger than numerical error. States 13, 14 and 15 share cost 676.670168 because dispatch empties the station: $J^*(13) = J^*(14) = J^*(15) = 100 + J^*(0)$.

1.2 Edge cases and scientific validation

The following experiments make predictions before checking the implementation; they test the MDP and algorithm, beyond input-format checks.

Prediction / edge case	Quantitative evidence and handling
Independent small decision tree must agree with dynamic programming	Literal adaptive-tree enumeration over 7 states and horizons 0–4 agrees within numerical tolerance; it does not call production transition/Bellman routines.
A returned value must satisfy Bellman optimality	PI residual 4.55×10^{-13}; VI/PI differ by 9.62×10^{-9}, with zero action disagreements. An independent certificate gives residual 9.09×10^{-13}.
Empty, partial and over-capacity service	Waiting at 0 saves exactly 100. Dispatch at 13 serves 13; dispatch at 200 serves 15, leaves 185, costs 470, then reaches 186–190. No negative queues or unlimited service.
The upper boundary must aggregate probability	Waiting at 198 reaches 199 with probability 0.2 and 200 with probability 0.8; waiting at 200 stays at 200. Under the assigned optimum no arrivals clip; expanding the cap to 300 changes costs on 0–200 by at most 4.55×10^{-13}.
Terminal value and extreme economics have known effects	At the final decision, wait everywhere: dispatch costs 100 but saves at most 30 holding cost. Zero holding cost favors waiting; free service favors dispatch. With no arrivals, costs at states 0–3 are 0, 40, 80, 100.
A threshold need not survive changed assumptions	With $K = 5$, free overflow rejection makes dispatch optimal at 5–192, then wait at 193–200. The solver is not constrained to a threshold policy.
Simulation must match exact policy values	Independent simulation uses 20,000 trajectories, 400 periods, common random arrivals and seed 3817. Policy means are within 1.565 estimated SE of exact values; deterministic always-dispatch agrees with its truncated analytic value within 2.3×10^{-12}.

Independent reconstruction. `verify_q1_baseline.py` reconstructs all 100,701 finite-horizon values (maximum difference 3.18×10^{-12}) and 100,500 actions (zero disagreements), checks all 202 threshold policies, and certifies the infinite optimum. `verify_q1_exhaustive.py` enumerates all 1,920 stationary policies across 30 small models, checking VI and PI without assuming a threshold. Evidence: results/q1/validation and results/q1/verifier.

Ten-step deterministic diagnostic. Replacing random arrivals by exactly 3 gives states $0, 3, 6, 9, 12, 15, 3, 6, 9, 12$, actions $0, 0, 0, 0, 0, 1, 0, 0, 0, 0$, costs $0, 6, 12, 18, 24, 100, 6, 12, 18, 24$, and next states $3, 6, 9, 12, 15, 3, 6, 9, 12, 15$. At state 15, dispatch empties the queue and $669.753019 = 100 + 0.95(599.740020)$. The full state/value/reward trace is saved in deterministic_first_10_steps.csv.

Three more months. First strengthen independent solver/property checks and near-tie numerical certificates. Then vary overflow penalties, correlated or bursty arrivals, partial service and shuttle availability, expanding the state when needed for Markov sufficiency. Finally estimate robust policy performance under misspecified demand, with larger simulation sets, tail-cost measures and independent confidence intervals. These are proposed experiments, not completed results.

2. Policy Gradient: Part 1

Implement **basic PyTorch REINFORCE without a baseline**. For full episodes, $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$, with zero continuation after the last observed reward. For $B = 16$ complete episodes under frozen parameters:

$$J(\theta) = \mathbb{E} \sum_t \gamma^t r_t, \qquad L_1 = -\frac{1}{Bc} \sum_{i=1}^{B} \sum_t \gamma^t G_{i,t} \log \pi_\theta(a_{i,t} \mid x_{i,t}).$$

Returns are detached. The outer γ^t is required for this start-discounted objective. Action terms are summed, not divided by episode length. Positive fixed scales $c = 20$ (CartPole) and 1 (Pong) change gradient units, not reward baselines. Adam epsilon makes scaling imperfectly equivalent to a learning-rate change. There is no centering, advantage normalization, entropy bonus, critic or reward shaping.

Setting	CartPole-v1	ALE/Pong-v5 (namespaced Pong-v5)
Discount / network	0.95; 4 → 64 tanh → 2 logits	0.99; 6,400 → 64 ReLU → 2 logits
Adam / training	0.003; 6,000 episodes	0.01; 10,000 episodes
Inputs / actions	Four original state coordinates; 0/1	Assignment-preprocessed frame differences; logits 0/1 map only to actions 2/3

Pong crops rows 35–194, downsamples by 2, removes background colors 144/109, binarizes and subtracts the previous 80×80 frame. Reset clears history, so the first difference is zero. This is partial observation, not a claim of a fully Markov input. Official v5 frame skip 4, sticky probability 0.25 and episode limit are retained; no extra FIRE/no-op wrapper or point-boundary discount reset. All reported Deepdish training uses one RTX PRO 5000; CartPole main runs use local CPU. CPU environment workers feed the GPU, without multi-GPU training.

Training-only 3,000-episode CartPole pilots at Adam 0.01/0.003/0.001 had last-100 means 95.82/252.87/82.08; 0.003 was selected before the fresh 6,000-episode run. It still deteriorated: best 100-window 290.36 at episode 3,097; final window 166.72. Pong retained the requested 0.01. No final evaluation selected rates or checkpoints.

2.1 Required training and 500-game evaluation plots

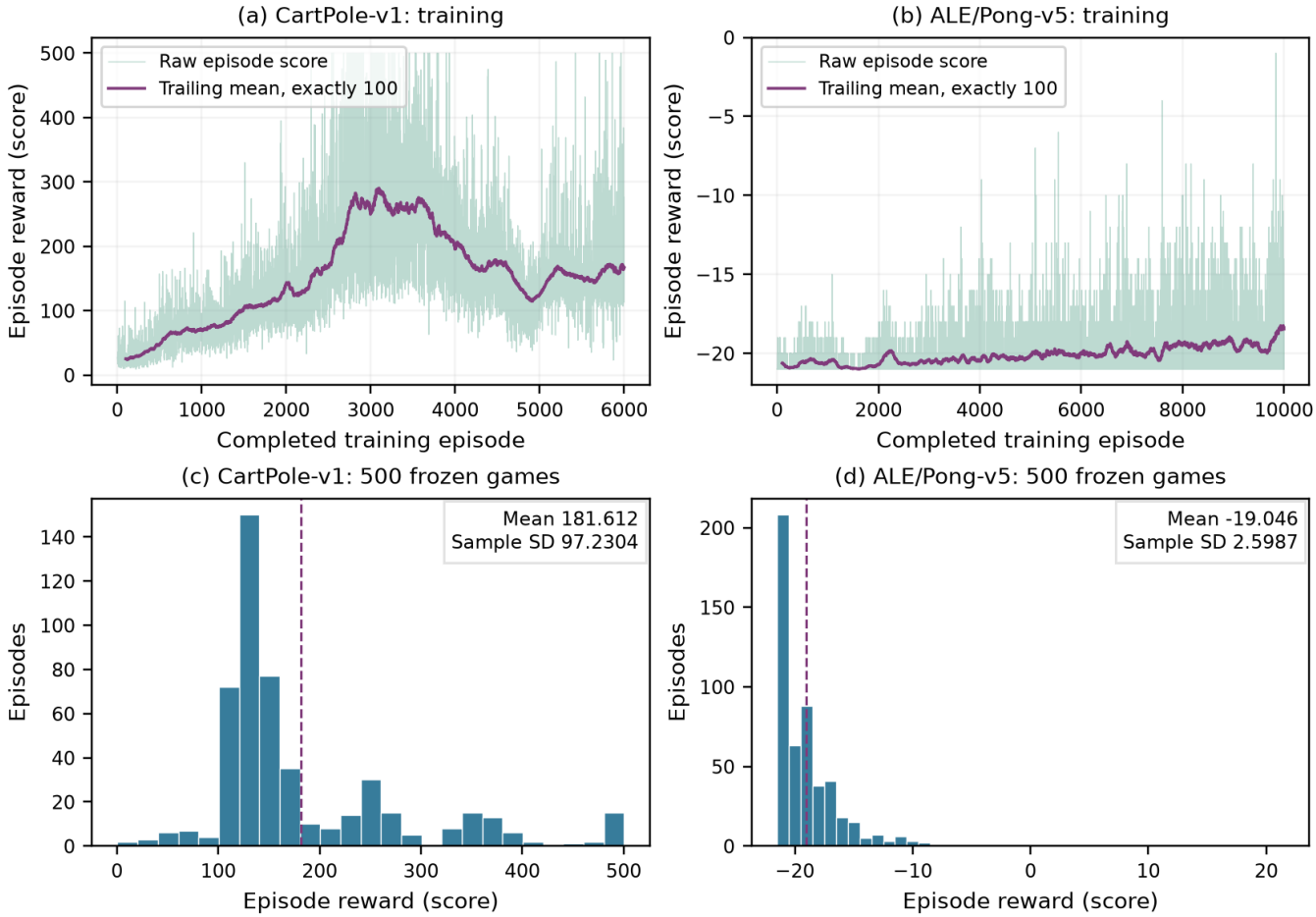


Figure 2. (a–b) Raw training scores and a trailing average of **exactly 100** episodes, beginning at episode 100. (c–d) Histograms of **500 frozen stochastic** evaluation episodes per game. Final, not hindsight-best, checkpoints are tested. All means and sample SDs below are independently recomputed from episode CSVs.

Frozen Part 1 policy	Test games	Mean	Sample SD	Range / outcome
CartPole, seed 3817	500	181.612	97.2304	20–500; 15 reach 500
Pong, seed 3817	500	-19.046	2.5987	-21 to -3; 0 wins
CartPole uniform reference	500	22.140	12.7489	Separate diagnostic

Approximate 95% mean intervals are [173.0894,190.1346] for CartPole and [-19.2738,-18.8182] for Pong, conditional on these frozen policies. All 500 Pong games terminate naturally, without truncation. Three CartPole training seeds 3817/3818/3819 yield evaluation means 181.612/131.218/174.812 (each 500 games): mean of means 162.5473, across-seed sample SD 27.3442. Reset banks overlap, so these are descriptive replications, not an independent-seed confidence interval.

Interpretation. CartPole improves over random but is not reliably solved. Pong remains poor despite modest score improvement: first/last 500 training means -20.742/-19.142 and final policy entropy about 0.1065. Fresh 64-game diagnostics give trained/uniform/always-RIGHT/always-LEFT means -18.6563/-20.5781/-21/-21, all with zero wins. This detects some learned behavior but no competent winning policy. Sparse credit, partial observation and action concentration are plausible limitations, not established causes. Lower rates are not a demonstrated Part 1 fix.

2.2 Task-specific correctness, edge cases and limitations

Experiment / prediction	Quantitative result and handling
Exact multistep objective must match the gradient estimator	Enumerated trajectory probabilities and direct differentiation agree with REINFORCE within 2.78×10^{-17}. An independent variable-length finite-difference oracle agrees within 1.57×10^{-11}. This checks discount placement and stopping, rather than trusting a rising curve.
Known-optimum noisy bandit must learn the better action	Reward probabilities 0.2/0.8 give uniform value 0.5. Three runs learn best-action probabilities 0.97787, 0.97897 and 0.97974, with expected rewards about 0.787.
Signed, zero, delayed and boundary-discount rewards	For [1,-2,3], gamma 0.5 gives returns [0.75,-0.5,3] and objective weights [0.75,-0.25,0.75]. Gamma 0 credits only step 0; gamma 1 recovers future sums; all-zero rewards yield zero gradient. Punishing an action lowers its probability from 0.5 to 0.49500 after Adam.
Pong points must not reset discounting	[1,0,-1] at gamma 0.99 gives returns [0.0199,-0.99,-1], retaining the next point. A reward delayed 1,000 steps has weight $0.99^{1000} = 4.3171 \times 10^{-5}$. Float64 retains small values but cannot prevent vanishing credit.
Physical dynamics and episode limits must match predictions	Independent CartPole equations agree within 1.71×10^{-9}. A diagnostic one-step TimeLimit has length/return 1. The actual task keeps its official cap. Stop on either termination or truncation, retain the last reward, and clear frame history. Flags can both be true.
Image/action restrictions and complete games	Synthetic preprocessing preserves exactly the predicted foreground/motion pixels without mutating input. A real Pong game using only 2/3 ends after 888 decisions with score -21, agent/opponent points 0/21. Official settings and labels are checked in the emulator.
A restart or faster collector must not alter the claimed computation	Interrupted 8→16-episode runs reproduce uninterrupted parameters exactly on CPU and on the single Deepdish GPU. Independent chunked/unchunked Adam updates agree within 9.32×10^{-10}. Partial batched trajectories and score-only evaluation reproduce serial states/actions/rewards; worker-count changes reproduce fast CPU runs.

First ten decisions. In the saved uniform-policy CartPole diagnostic, all rewards are +1, sampled actions are 1,0,1,1,1,1,1,1,0,0, and state, next state, probability, return and gradient weight are recorded in results/q2/validation/cartpole_first_10_steps.json. This trace and the physics oracle expose state/action/reward interactions; see the Part 2 trace below for the effect of a nonzero baseline.

Why score can remain weak. With gamma 0.95, constant-reward CartPole returns are 19.88159 after 100 steps and almost 20 after 500. Pong's outer discount is about 1.50×10^{-22} at step 5,000. Correct optimization of the specified discounted objective does not guarantee high undiscounted scores. The required objective is preserved instead of silently resetting credit at each point.

Three more months. Run preregistered rate/batch-size and observation ablations with independent training seeds and disjoint test banks. Measure entropy, point timing, truncations and gradient variance throughout learning. Benchmark full update throughput and GPU replay; use a separate validation set for any checkpoint-selection rule, then one untouched final 500-game test.

3. Policy Gradient: Part 2 — repeat with a baseline

Use the same games, actor networks, discounts, rates, budgets and stochastic evaluation as Part 1, training fresh policies. The chosen **constant-within-batch baseline** is a startup-corrected EMA of previous batches' episode-start returns. Initialize $n_0 = z_0 = 0, b_0 = 0$; freeze b_k before sampling batch k. After the actor step, with batch mean m_k of G_0, update

$$n_{k+1} = 0.9n_k + 0.1m_k, \quad z_{k+1} = 0.9z_k + 0.1, \quad b_{k+1} = n_{k+1}/z_{k+1},$$

$$L_2 = -\frac{1}{Bc}\sum_{i,t}\gamma^t(G_{i,t} - b_k)\log\pi_\theta(a_{i,t} \mid x_{i,t}).$$

The current episode never fits its own baseline. Conditional on past data and the current state, b_k is fixed and $\mathbb{E}_a[\nabla \log\pi(a \mid s)] = \sum_a \nabla\pi(a \mid s) = 0$. Thus subtraction preserves the expected gradient, including variable-length episodes. It can change variance and Adam steps. Returns/baseline are detached; there is no bootstrap, standardization, entropy bonus or reward reshaping. This return-mean constant is simple and cheap, not generally variance optimal (Greensmith et al., 2004).

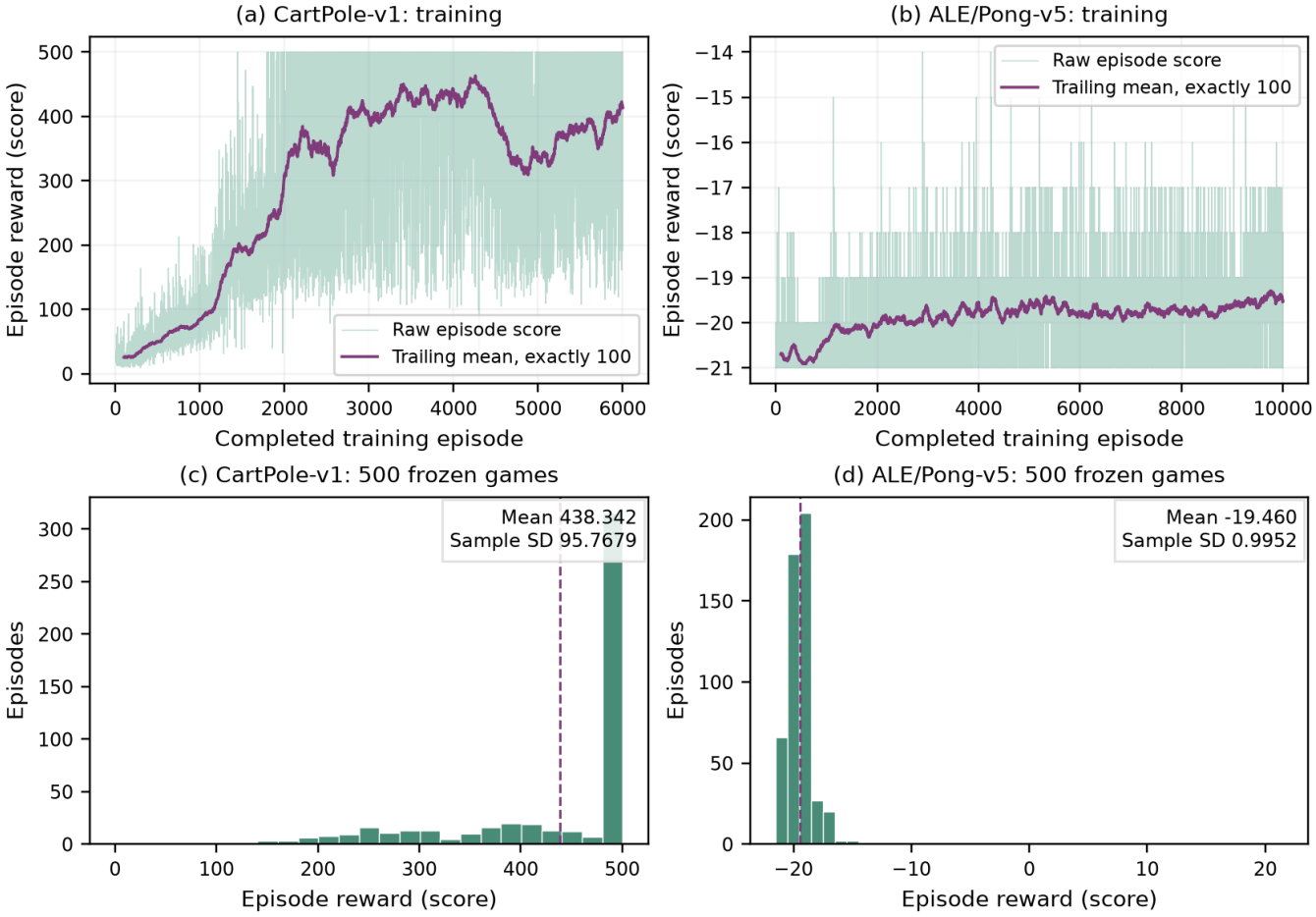


Figure 3. The repeated CartPole/Pong experiments: (a–b) raw scores with exact 100-episode trailing means; (c–d) the final trained stochastic policies' 500-game histograms. Training budgets are 6,000 and 10,000 complete episodes respectively.

3.1 Results, variance and failure cases

Training seed	Part 1 CartPole mean / SD	Part 2 CartPole mean / SD	Part 2 at 500
3817	181.612 / 97.2304	438.342 / 95.7679	303/500
3818	131.218 / 23.3961	434.470 / 102.0657	301/500
3819	174.812 / 59.8051	388.232 / 104.6479	152/500

Each row evaluates 500 games per policy. Part 2's primary CartPole range is 46–500, approximate 95% mean interval [429.9476,446.7364]. The three-seed mean of means rises from 162.5473 to 420.3480; across-seed SD rises slightly from 27.3442 to 27.8806. This supports better scores, not reduced between-training variability. Gradient variance and score SD are different quantities.

Pong Part 2: mean **-19.460**, sample SD **0.9952**, range -21 to -15, approximate 95% mean interval [-19.5472,-19.3728], **0/500 wins**, 500 natural terminations and zero truncations. Its discounted mean +0.12595 exceeds Part 1's -0.26569 even though its game score is worse. A valid baseline and reduced gradient variance did not yield a winning policy. The comparison uses one Pong training seed and includes a collector/RNG transition in Part 1, limiting causal claims.

Direct frozen-policy variance tests. Fit the baseline on independent calibration episodes, freeze it, and compute both gradients on the **same** probe trajectories. Measure trace of sample gradient covariance (ddof=1), with 2,000 paired-bootstrap resamples; ratio below 1 means reduction.

Frozen policy	Calibration / probes	Raw variance	With baseline	Ratio [95% interval]
Initial CartPole	64 / 128	0.97880	0.94067	0.9610 [0.7148,1.2498]
Final Part 1 CartPole	64 / 128	18.08717	0.30767	0.01701 [0.0000615,0.05208]
Initial Pong	32 / 32	124.50566	15.40717	0.12375 [0.09414,0.16511]

The initial CartPole result is inconclusive; final CartPole and initial Pong show about 98.3% and 87.6% reduction. Intervals condition on the fixed policy and fitted baseline, not training-seed or calibration uncertainty. Different network dimensions/scales prevent meaningful comparison of absolute variances across games.

Exact predictions and counterexamples. Enumerating eight fixed-length and five variable-length paths shows baselines -10, 0 and 5 preserve the direct objective gradient within 10^{-12}; an action-dependent baseline produces error 0.24026. In a Bernoulli bandit with $p = 0.3, r = a$, the true gradient is 0.21, but current-batch mean subtraction yields 0 at batch size 1 and 0.1575 at size 4: the bias factor is $1 - 1/B$. The lagged update avoids this dependency. The return-mean baseline reduces deterministic-bandit variance 0.1029→0.0336, while the optimal constant makes it numerically zero. In a noisy bandit with $p = 0.98$, it instead **increases** variance 0.00401690→0.00963881. Baselines are not guaranteed to help. The noisy 0.2/0.8 bandit still learns best-action probabilities 0.98007/0.97784/0.97948 across three seeds.

3.2 Baseline edge cases and a traced actor update

Startup correction maps a first target 20 to baseline 20, not 2. Zero targets keep zero without standard-deviation division; negative targets give negative baselines. Gamma 0 credits only the initial action. A stale baseline 2 on a zero-reward bandit can add variance 0.84 while preserving expected gradient zero. The first batch uses $b = 0$ and exactly matches Part 1's parameters after Adam. Stopped/resumed training reproduces policy, baseline state and optimizer diagnostics; no current action influences its own baseline fit.

Ten-step example. The first CartPole episode after the initial 16-game batch uses the frozen baseline **13.6720064**. Initial state (position, velocity, pole angle, angular velocity) is approximately (-0.001779,-0.042984,-0.007022,-0.012988). Every displayed reward is +1. Probability changes reflect state changes, not in-episode parameter updates.

Step	Pole angle	Action	P(action)	G_t	$G_t - b$	$\gamma^t(G_t - b)/20$
0	-0.00702	0	0.5015	15.4813	1.8093	0.09046
1	-0.00728	1	0.5019	15.2435	1.5715	0.07464
2	-0.00173	0	0.5016	14.9931	1.3211	0.05962
3	-0.00208	1	0.5018	14.7296	1.0576	0.04534
4	0.00341	0	0.5016	14.4522	0.7802	0.03177
5	0.00304	1	0.5018	14.1602	0.4882	0.01889
6	0.00854	0	0.5016	13.8529	0.1809	0.00665
7	0.00820	0	0.4981	13.5293	-0.1427	-0.00498
8	0.01378	0	0.4947	13.1888	-0.4832	-0.01603
9	0.02526	1	0.5087	12.8303	-0.8417	-0.02652

At step 7, the return drops below the baseline despite a positive reward, so this term discourages the sampled action. The eventual optimizer step sums all decisions and episodes; one term's sign does not determine the final parameter change. Full states, next states and flags are in results/q3/cartpole/final_seed3817/first_10_steps_after_baseline.json.

Runtime and evidence audit. CPU environment workers collect every requested complete episode, including slow ones, under frozen actor parameters. They do not replace early finishers mid-batch. Eight workers with one GPU measured 13,305 decisions/s versus 3,007 for serial GPU inference in a matched 32-game benchmark; this is rollout throughput, not guaranteed end-to-end speedup. Part 1 switches from serial collection after episode 5,376 to independent per-episode action streams; previous history and a source/checkpoint snapshot are preserved, without claiming bitwise replay of the old sampling stream. The saved-artifact verifier reconstructs all 2,030 baseline updates across the main/follow-up checkpoints within 1.43×10^{-14}, checks 80 traced decisions and matches evaluation hashes. This consistency audit supplements, not replaces, the scientific tests.

Three more months. First repeat frozen-policy variance experiments at multiple training stages with larger independent calibration/probe sets and disjoint seed banks. Then compare decay, time-dependent and state-value baselines fitted on past/cross-fitted data, including stale-baseline and rare-failure counterexamples. Finally preregister held-out score, win-rate, variance and compute comparisons across more seeds. Analyze early/late Pong point credit without changing the assigned discount or quietly replacing the required algorithm.

4. Execution, provenance and submission checks

Use Python 3.11+; the recorded runs use Python 3.13, PyTorch 2.15.0, Gymnasium 1.4.0, ALE 0.12.1, NumPy 2.5.4 and Matplotlib 3.11.2. Per-run metadata records exact versions. From the public repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-q2.txt
python q1_shuttle.py
python validate_q1.py
python verify_q1_baseline.py
python verify_q1_exhaustive.py
```

For the main **local CPU** CartPole experiments, run the following for `N=2` (Part 1) and `N=3` (Part 2), each with a fresh directory. Add `--seed 3818` or `--seed 3819` to train for the other replications. Evaluation reads the checkpoint configuration.

```
N=2
python q${N}_policy_gradient.py train --game cartpole --episodes 6000 \
  --lr 0.003 --output-dir results/q${N}/cartpole/reproduction
python q${N}_policy_gradient.py evaluate --episodes 500 \
  --checkpoint results/q${N}/cartpole/reproduction/checkpoint.pt \
  --output-dir results/q${N}/cartpole/reproduction/evaluation_trained
```

Deepdish: exactly one GPU. Install a compatible CUDA PyTorch build and expose your assigned card. CPU workers emulate games; neural inference/updates use CUDA. To reproduce the reported Part 1 serial-to-batched boundary and the fresh Part 2 Pong experiment, use fresh directories and these commands:

```
export CUDA_VISIBLE_DEVICES="your-assigned-GPU-UUID"
export CUBLAS_WORKSPACE_CONFIG=:4096:8
python q2_policy_gradient.py train --game pong --episodes 5376 --lr 0.01 \
  --reward-scale 1 --device cuda --output-dir results/q2/pong/reproduction
python fast_pg.py train --part 1 --game pong --episodes 10000 --lr 0.01 \
  --reward-scale 1 --device cuda --workers 8 \
  --resume results/q2/pong/reproduction/checkpoint.pt \
  --output-dir results/q2/pong/reproduction
python fast_pg.py train --part 2 --game pong --episodes 10000 --lr 0.01 \
  --reward-scale 1 --device cuda --workers 8 \
  --output-dir results/q3/pong/reproduction
```

For each Pong directory, evaluate with `python fast_pg.py evaluate --device cuda --workers 8 --episodes 500 --checkpoint <directory>/checkpoint.pt --output-dir <directory>/evaluation_trained` (one command). Targets are **total** episodes; resume requires matching actor settings. The default baseline for Part 2 is EMA 0.9. An all-serial Part 1 rerun uses a different action stream after episode 5,376.

Scientific validation commands (run locally). `python validate_q2.py --pong` runs the exact gradient, known-optimum learning, signed/sparse reward, physics, limit and replay tests. `python validate_q3.py --variance --pong` adds baseline invariance/bias, variance counterexamples, learning, replay and independent-calibration frozen gradient measurements. `python verify_q2_independent.py` checks production updates and saved CartPole replay; `python verify_q2_completed_collector.py` checks full versus score-only Pong collection; `python verify_q3_completed.py` audits saved baseline/evaluation evidence. Use each validator's `--output-dir` option where available to preserve supplied evidence. `python validate_fast_pg.py` tests reset, partial-batch, worker-count and checkpoint equivalence.

Required results are in results/q1, results/q2 and results/q3. Checkpoints, CSVs, histograms, traces, source hashes and expanded notes remain on GitHub. Separate exploratory studies do not replace the required Part 1/Part 2 comparison. `python report_figures.py` regenerates the 11 required plots as three composite figures; `python build_report.py` builds this PDF from report/submission.md. No new long training is needed to inspect the supplied results. The final manifest records artifact checksums.